

Asignatura:	Sistemas distribuidos	Carrera y semestre:	ING. EN SISTEMAS COMPUTACIONALES OCTAVO SEMESTRE		Plan:	ISIC-2010-224	
No. de tema:	3,4,5	Nombre del tema:	Comunicación y asignación de Nombres			Fecha:	
Nombre estudiante:			Firma:			Calificación:	
Tipo de evaluación (Marque con una X)							
Curso ordinario		Curso de repetición		Curso ordinario semipresencial		Curso especial	
Evaluación de primera oportunidad	Evaluación de segunda oportunidad	Evaluación de primera oportunidad	Evaluación de segunda oportunidad	Evaluación de primera oportunidad	Evaluación de segunda oportunidad	Evaluación de primera oportunidad*	Evaluación de segunda oportunidad*

INSTRUCCIONES: Diseñarán e implementarán un Sistema de Alerta Ciudadana distribuido tipo C5, simulando la infraestructura real de centros de comando y control de emergencias. El sistema recibirá alertas desde dispositivos físicos (ESP32 con botón de pánico), las procesará a través de microservicios independientes y notificará a operadores en tiempo real.

CONTEXTO DEL PROBLEMA: Un ciudadano en situación de emergencia presiona un botón físico conectado a un ESP32. El dispositivo captura su ubicación GPS y publica una alerta vía MQTT. El sistema distribuido debe recibirla, clasificarla por prioridad, asignar unidades de respuesta y notificar a los operadores en tiempo real, garantizando que ninguna alerta se pierda, incluso si uno o más nodos fallan.

ARQUITECTURA REQUERIDA: El sistema debe estar compuesto por los siguientes 5 microservicios independientes, cada uno con su propio proceso/contenedor:

#	Microservicio	Responsabilidad
1	Recepción de alertas	Suscripción MQTT, validación y encolado de mensajes entrantes
2	Geolocalización	Procesamiento de coordenadas GPS, zona geográfica y mapeo
3	Asignación de Prioridad	Clasificación de alertas (crítica / alta / media) según reglas configurables
4	Notificaciones	Envío en tiempo real a operadores vía WebSockets
5	Historial de Incidentes	Persistencia, consulta y auditoría de todas las alertas procesadas

STACK TECNOLÓGICO OBLIGATORIO

- Dispositivo: ESP32
- Protocolo IoT: MQTT (broker Mosquitto o HiveMQ)
- Comunicación entre servicios: REST o gRPC (mínimo un servicio debe usar gRPC)
- Tiempo real: WebSockets para notificaciones al operador
- Caché / cola en memoria: Redis (pub/sub o colas de tareas)
- Persistencia: PostgreSQL con al menos una réplica de lectura configurada
- Contenedores: Docker + Docker Compose (obligatorio)

REQUISITOS FUNCIONALES

#	Requisito Funcional
1	El sistema debe recibir alertas del ESP32(mínimo 2) vía MQTT y procesarlas en menos de 2 segundos end-to-end.
2	Cada alerta debe contener: ID de dispositivo, coordenadas GPS (lat/lon), timestamp y tipo de emergencia.

3	El microservicio de prioridad debe clasificar automáticamente las alertas en tres niveles: crítico, alto y medio, con reglas documentadas.
4	Los operadores deben recibir notificaciones en tiempo real vía WebSocket con los datos completos de la alerta.
5	El historial debe ser consultable por rango de fechas, zona geográfica y nivel de prioridad.
6	El sistema debe soportar al menos 3 instancias del servicio de Recepción de Alertas corriendo en paralelo detrás de un balanceador de carga (Nginx o Traefik).

REQUISITOS NO FUNCIONALES (Distribuidos)

#	Requisito no funcional
1	Tolerancia a fallos: Si el microservicio de Notificaciones cae, las alertas NO deben perderse. Deben encolarse en Redis y entregarse al recuperarse el servicio. El equipo debe demostrarlo en vivo.
2	Balanceo de carga: Las alertas entrantes deben distribuirse entre las instancias del servicio de Recepción mediante least-connections. Debe mostrarse en la demo con logs.
3	Consistencia y replicación: PostgreSQL debe tener configurada replicación maestro-réplica. Las lecturas del historial deben dirigirse a la réplica. Documentar el modelo de consistencia elegido (eventual vs. fuerte) y justificarlo.
4	Comunicación entre servicios: Al menos un par de microservicios debe comunicarse mediante gRPC con contrato .proto definido. El resto puede usar REST con OpenAPI documentado.

ENTREGABLES:

1. Repositorio en Github o similar.
 - a. Archivo docker-compose.yaml (debe levantar todo el sistema)
 - b. Readme.md (Instrucciones completas y claras para correr el sistema y usarlo)
 - c. Carpeta services (contiene todos los microservicios)
 - d. Carpeta esp32 (contiene los archivos de código de los dispositivos de hardware)
 - e. Carpeta Docs (contiene las documentaciones de los servicios, diagrama de arquitectura, etc.)
2. Demo en vivo.

EVALUACIÓN:

1. Criterios de aprobación mínima.
 - a. Completar obligatoriamente la demo en vivo (si algo falla o no presenta algo obtiene 0 en el examen)
 - b. Todos los integrantes deben estar presentes a la hora de la entrega.
2. Penalizaciones:
 - a. -10 puntos si no levanta con Docker en un solo comando
 - b. -5 por cada microservicio que falle, deba reiniciarse manualmente o configurarse al momento.
 - c. -50 por código copiado entre equipos o que no sepan su funcionamiento.
3. Rubrica de evaluación:

#	Categoría	Criterio	Excelente	Suficiente	Insuficiente
1	Arquitectura y Diseño (20 pts)	Diagrama de arquitectura	Completo, claro, con todos los componentes y flujos (10 pts)	Presente pero incompleto o sin flujos de datos (6 pts)	Ausente o incomprensible o generado con IA (0 pts)



INSTRUMENTO DE EVALUACIÓN

FO-ACA-13

versión 1



2		Registro de decisiones ADR	Documenta ≥3 decisiones con contexto, alternativas y justificación (10 pts)	Documenta decisiones pero sin alternativas ni justificación (5 pts)	Ausente o genérico (0 pts)
3	Implementación de Microservicios (25 pts)	Los 5 microservicios implementados y funcionales	Los 5 corren y cumplen su responsabilidad (15 pts)	3–4 servicios funcionales (9 pts)	Menos de 3 funcionales (0 pts)
4		Comunicación gRPC con contrato proto	Implementado y funcional entre ≥2 servicios (10 pts)	Implementado, pero con errores parciales (5 pts)	No implementado (0 pts)
5	Tolerancia a Fallos (20 pts)	Caída y recuperación sin pérdida de alertas	Demostrado en vivo con evidencia en Redis y logs (20 pts)	Demostrado, pero con pérdida parcial o sin evidencia clara (10 pts)	No se demuestra (0 pts)
6	Balanceo de Carga (15 pts)	Múltiples instancias con balanceador activo	3+ instancias funcionales, distribución visible en logs (15 pts)	2 instancias o balanceo no verificable (8 pts)	No implementado (0 pts)
7	Replicación y Consistencia (10 pts)	PostgreSQL maestro-réplica, lecturas en réplica	Funcional, documentado y modelo de consistencia justificado (10 pts)	Configurado pero sin dirigir lecturas a réplica o sin justificación (5 pts)	No implementado (0 pts)
8	Repositorio y Reproducibilidad (5 pts)	Sistema levanta con docker-compose up sin errores	— (3 pts)	— (1 pt)	No levanta (0 pts)
9	Repositorio y Reproducibilidad (5 pts)	README claro con instrucciones completas	— (2 pts)	Instrucciones incompletas (1 pt)	Ausente (0 pts)
10	Defensa del proyecto (5 pts)	Respuesta técnica sobre el proyecto	Correcta, precisa y con criterio propio (5 pts)	Parcialmente correcta (3 pts)	Incorrecta o no responde (0 pts)